《操作系统综合实验》 实验报告

实验名称	多线程编程	实验序号	3
实验日期	2024.5.11	实验人	廖鑫洋

一、 实验题目

编写一个多线程排序程序，其工作方式如下：一个整数列表被分成两个大小相等的较小列表。两个单独的线程（我们将其称为 sorting threads）使用您选择的排序算法对每个子列表进行排序。这两个子列表随后由第三个线程——a merging thread——它将两个子列表合并为一个排序列表。

二、 相关原理与知识

1、冒泡排序和归并排序的过程与编程实践。

2、掌握<pthread.h>该库的基本用法，实现线程标识 pthread_t，pthread_create（）函数实现线程创建，pthread_join（）实现线程等待完成。

3、进程同步的相关知识。

三、 实验过程

1、输入需要排序的数组，以非数字字符结尾。

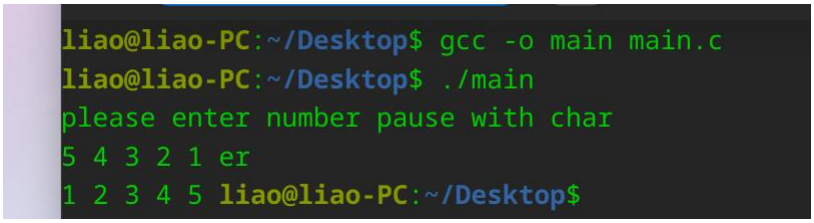
2、将输入的数组分成两部分，创建线程，每部分使用一个线程进行冒泡排序。

3、线程执行，按照冒泡排序子函数、分好的数组，分别在两个线程里面执行。

4、等待线程同步执行好后，使用归并排序将两个分别排序好的数组组合成一个排序好的数组。

5、输出排序好的数组。

6、最终运行结果，输入一系列数字字符，以非数字字符结尾，输出排序结果。



四、 问题总结

1、需要查找 pthread 库里面相关创建进程，进程标识，进程同步的操作，才能去实现多进程编程。

2、在写代码过程中可能会出现 bug，指针、数组等问题，认真调试即可。

五、 源代码

```
#include <stdio.h>
#include <pthread.h>

int *array;
void* sort(void* ptr){
    int *be=(int *)ptr;
    int *number=array;
    int begin=be[0];
    int end=be[1];
    for (int i = begin; i <end ; ++i) {
        for (int j = begin; j <end ; ++j) {
            if(number[j+1]<number[j]){
                int c=number[j];
                number[j]=number[j+1];
                number[j+1]=c;
            }
```

```
        }
    }
}
int main() {
    printf("请输入一系列用于排序的整数，以非整数字符结尾\n");
    int length=0;
    int a[100];
    array=a;
    while (scanf("%d",&a[length++]));
    /*  声明两个线程的描述符  */
    pthread_t tid1;
    pthread_t tid2;
    getchar();
    int be1[2]={0,((length-1)/2-1)};
    int be2[2]={((length-1)/2) ,(length-2)};
    pthread_create(&tid1, NULL, sort, be1);
    pthread_create(&tid2, NULL, sort, be2);
    pthread_join(tid1,NULL);
    pthread_join(tid2,NULL);
    int j=0,k=0,i=(length-1)/2;
    int answer[length-2];
    while(i<=length-2&&j<=(length-1)/2-1){
        if(a[j]>a[i]){
            answer[k++]=a[i++];
        } else{
            answer[k++]=a[j++];
        }
    }
    while (i<=length-2) answer[k++]=a[i++];
    while (j<=(length-1)/2-1) answer[k++]=a[j++];
```

```
    k=0;
    while (k<length-1){
        printf("%d ",answer[k++]);
    }
    return 0;
}
```